UF Herbert Wertheim College of Engineering
UNIVERSITY of FLORIDA

Electrical and Computer Engineering

Microcontroller DAC + SPI Module (Function Generator)

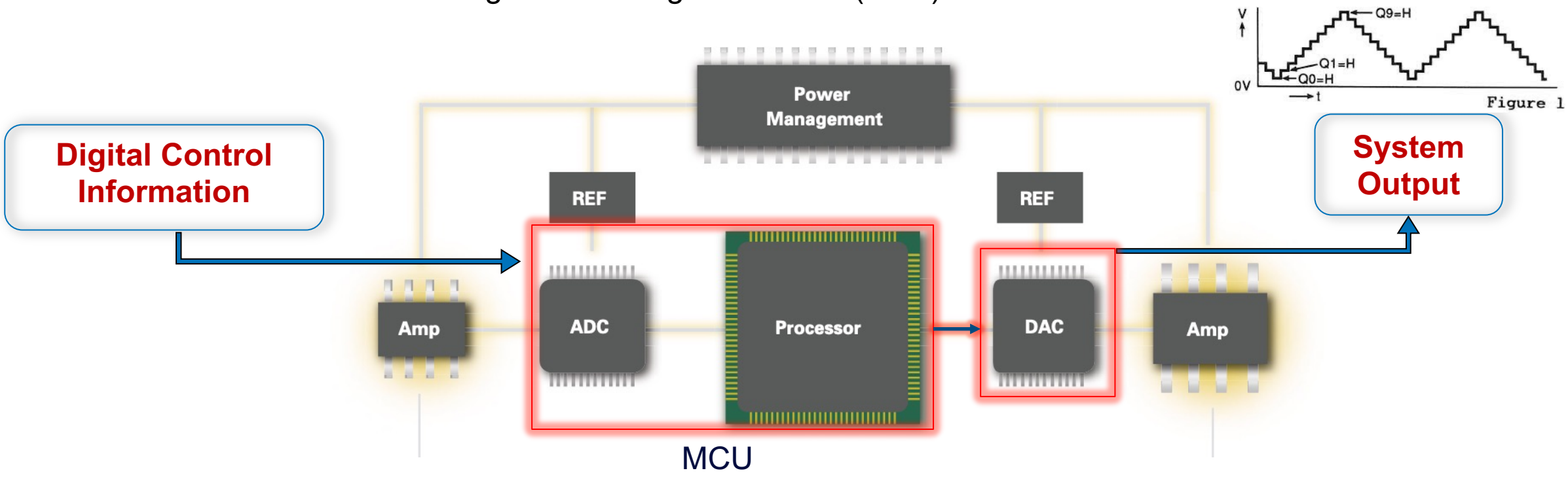
EEL3923C Design 1

Erin Patrick
Mike Stapleton

POWERING THE NEW ENGINEER TO TRANSFORM THE FUTURE

SPI + DAC Module Objectives

- Learn how to communicate to a peripheral device (e.g., DAC IC) with serial-peripheral interface (SPI) protocol on a microcontroller
- Learn about the limitations of digital-to-analog conversion (DAC) with a DAC IC



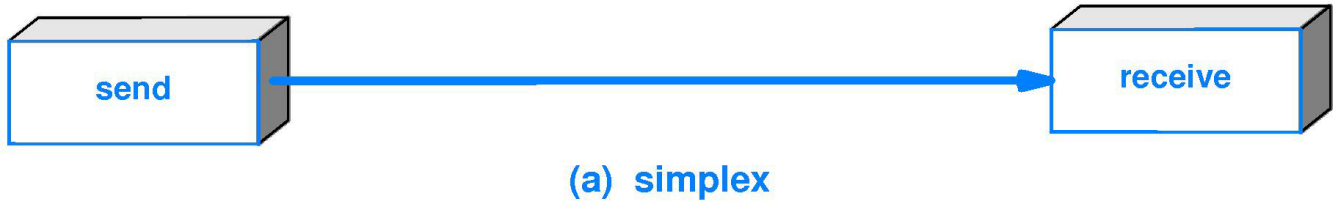
Serial Communication to Peripherals

Types of Serial Comm Protocols

Image source: https://circuitdigest.com/tutorial/serial-communication-protocols

- Serial Communication is a method of interfacing with more complex devices
  - Serial communication refers to the data being shifted in one bit at a time on one line
  - Parallel communication refers to data being sent in multiple bits at a time via multiple lines
  - Microcontroller to Microcontroller or Microcontroller to external device are examples using serial comm.
- There are multiple protocols that have been established over time, but the three we use frequently are I2C, SPI, and UART.
- Bit-banging is the process of emulating one of these protocols in software instead of dedicated hardware (setting pins to mirror the protocols instead of configuring the protocol)
  - It is preferrable to use the dedicated hardware in most cases, as we cannot guarantee that the processor will not be bottlenecked by the constant stream of data, the performance of the processor, or reliability of the processor.

Serial Communication Transmission Terminology



- In simplex transmission mode, the communication between sender and receiver occurs in only one direction.

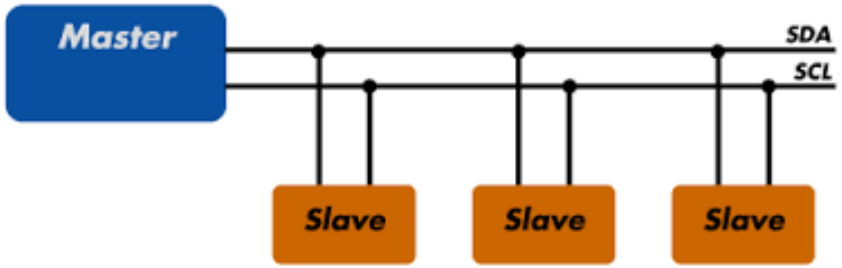
- In full-duplex transmission mode, the communication between sender and receiver can occur simultaneously.

- The communication between sender and receiver occurs in both directions in half-duplex transmission, but only one at a time.

source: https://www.black-box.de/en-de/page/25078/Resources/Technical-Resources/Black-Box-Explains/Fibre-Optic-Cable/simplex-vs-duplex-fiber-patch-cable

Serial Communication Protocol: I²C, I2C (Inter-Integrated Circuit)

- One bidirectional data line (SDA) and one shared clock line (SCL)
  - One device will transmit at a time (half duplex)
  - Transmissions will be ordered in a format shown in the table below
    - *All devices connected to these lines will receive this data and are differentiated by their address bits.*
    - *There is a start condition, r/~w bits for transmission direction, and acknowledge bits for error correction*



Connections between devices on the I2C lines

How each data transmission takes place with address, r/~w, acknowledge bits, and data

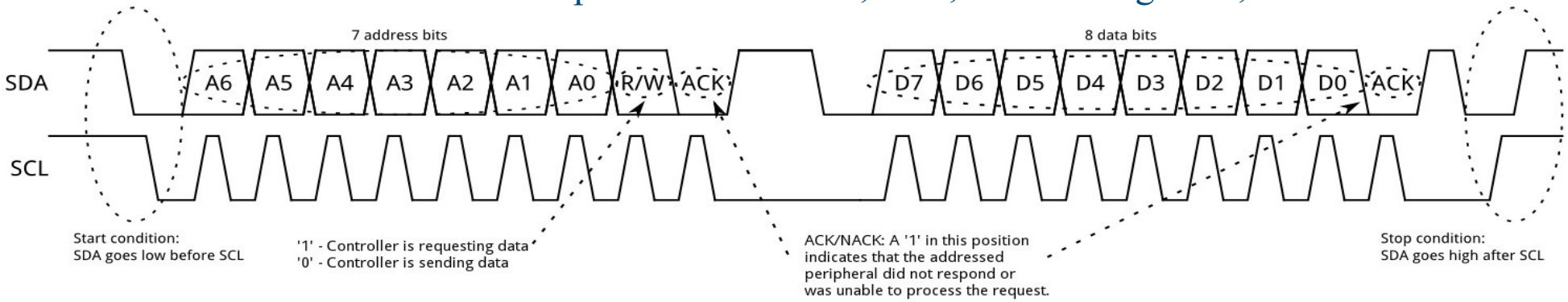


Image credit https://learn.sparkfun.com/tutorials/i2c/all

Serial Communication Protocol: SPI (Serial Peripheral Interface)

- Two Data lines – Output and Input
  - Usually called MOSI, MISO or SI, SO. From the reference point of the device being used. (MO – Master Output, SI – Slave Input). Full-duplex transmission
  - Depending on the application, MISO/SO lines can be omitted for one-directional communication (half duplex)
- One Clock line – CLK
  - Shared clock line provided by the master device
  - Clock speed must be within the allowance of the slave devices
- One Enable line – SS
  - This will determine which slave device is enabled before transmission
  - Each slave device connected to the MOSI, CLK lines will need its own SS signal to enable it for transmission

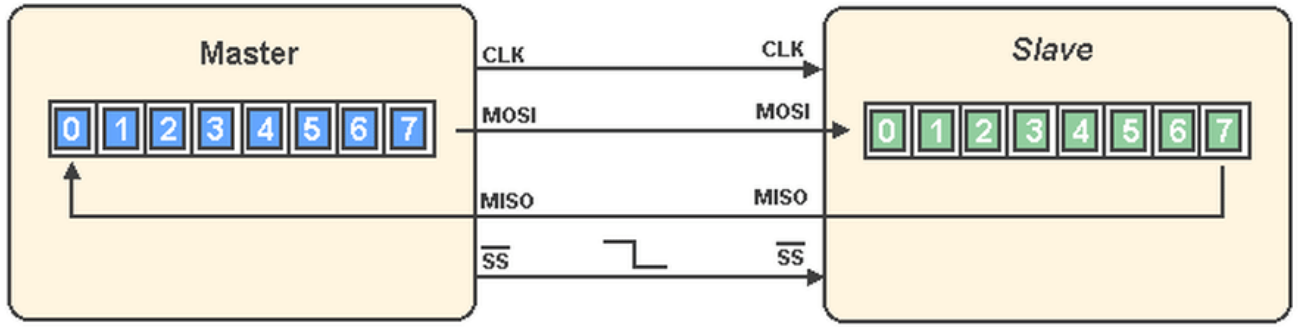
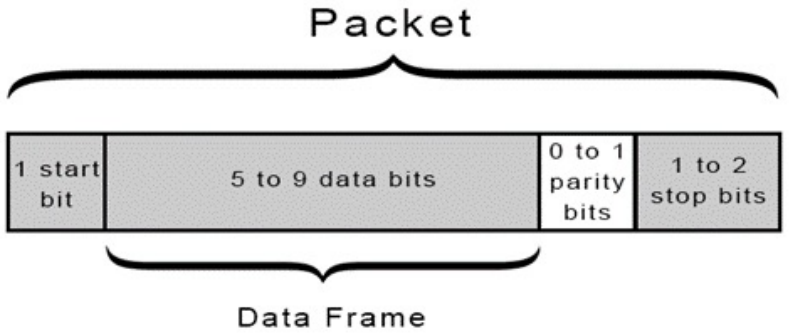


Image credit - https://gfycat.com/paltryillfossa

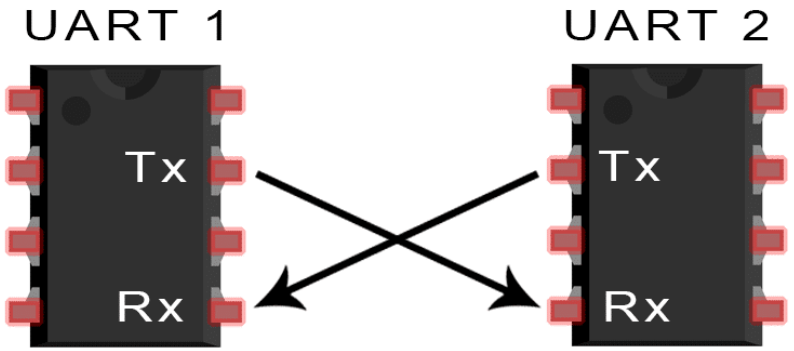
Showing how the data travels from device to device per clock edge, assuming the data transmitted in both devices is the same.

Serial Communication Protocol: UART

- Two data lines – Transmit (Tx) and Receive (Rx)
  - Both lines work asynchronously, meaning data can be sent at any time to the other device without a clock signal (full duplex)
  - Depending on the application, the Tx or Rx line can be omitted for one-directional communication
  - Due to the UART data packet (shown to the left) typically being 8 data bits large, it is common to send a character data byte.
- BAUD Rate
  - Because there is no shared clock line, in order to interpret data at the correct speed, both devices will need to be set with a matching BAUD (bits/second) rate
- Optional Error Detection/Correction
  - CTS (Clear-to-Send), RTS (Request-to-Send) lines may be needed if there is a high error rate in transmission.
  - Parity bits are used for error detection. It will not be able to correct the data, as it cannot determine which bit was incorrect.



A typical data "packet"



Showing how the data travels from device to device

Image credit - https://www.circuitbasics.com/basics-uart-communication/

Comparison of Common Protocols

I2C

- Uses fewer wires and connections
- Adding more controlled devices does not increase pins required
- Can have more than one controller (Master)
- All devices connected can communicate with each other regardless of designation (Master/Student)
- Slower than SPI

SPI

- Uses more wires and connections
- Adding more controlled devices increases the number of pins required proportionally – can only control as many devices as pins on the device will allow for
- Can only have one controller (Master)
- Devices will not be able to communicate with each other directly, but will have to go through the controller (Master)
- Much faster communication protocol than most alternatives

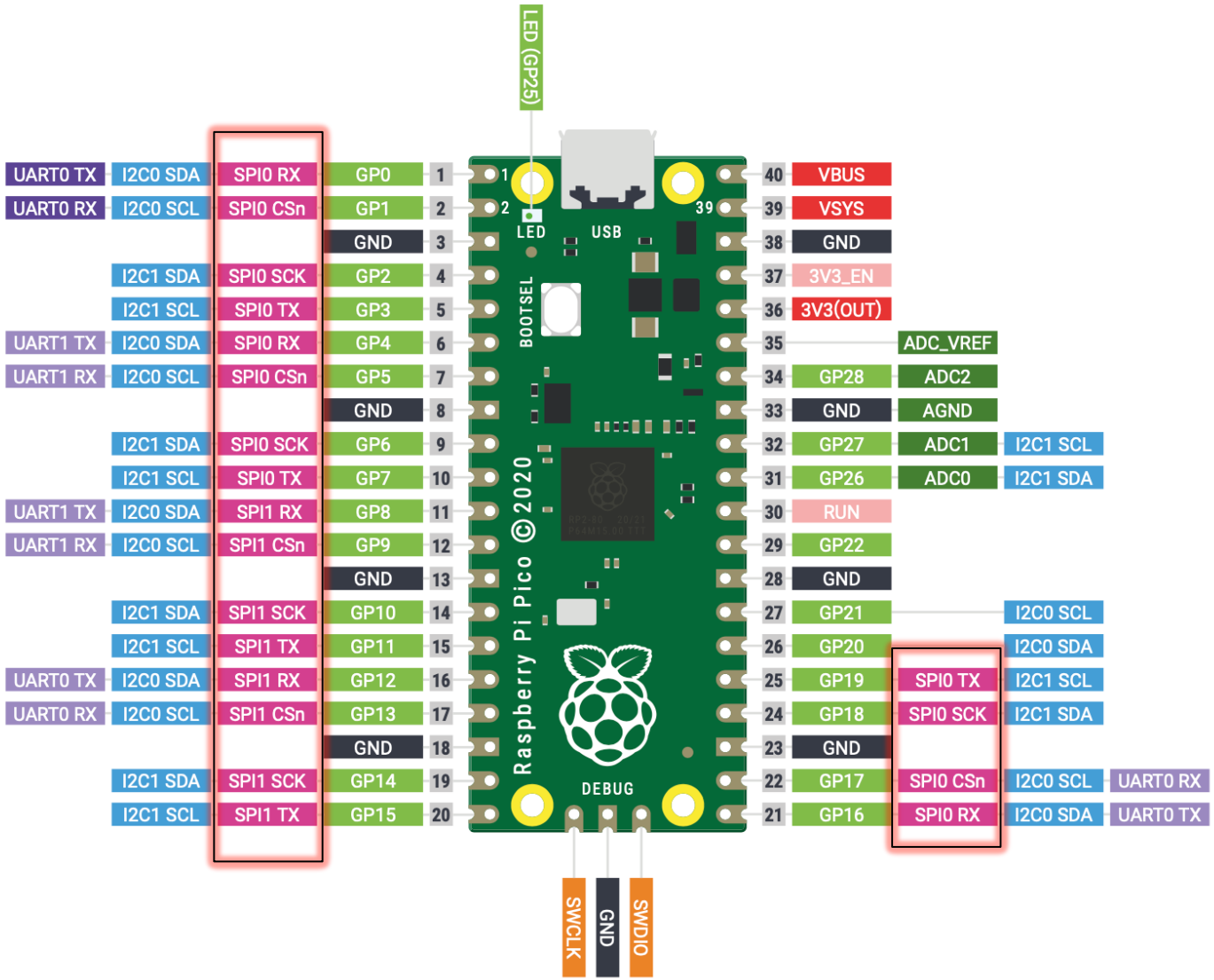
Used in this module

UART

- Can be sometimes unreliable
- Only supports communication with one other device
- Requires one to four wires depending on application
- Need to configure the communication speed between devices to match (BAUD rate)
- Slower compared to SPI and I2C

Pico Pins

- Only two SPI Registers: SPI0, SPI1



Pico MicroPython usage:

Example usage:

```
from machine import SPI, Pin

spi = SPI(0, baudrate=400000)


cs = Pin(4, mode=Pin.OUT, value=1)

try:
    cs(0)
    spi.write(b"12345678")
finally:
    cs(1)
```

Byte in decimal

Useful data structure:

buf=bytearray([HB, LB])

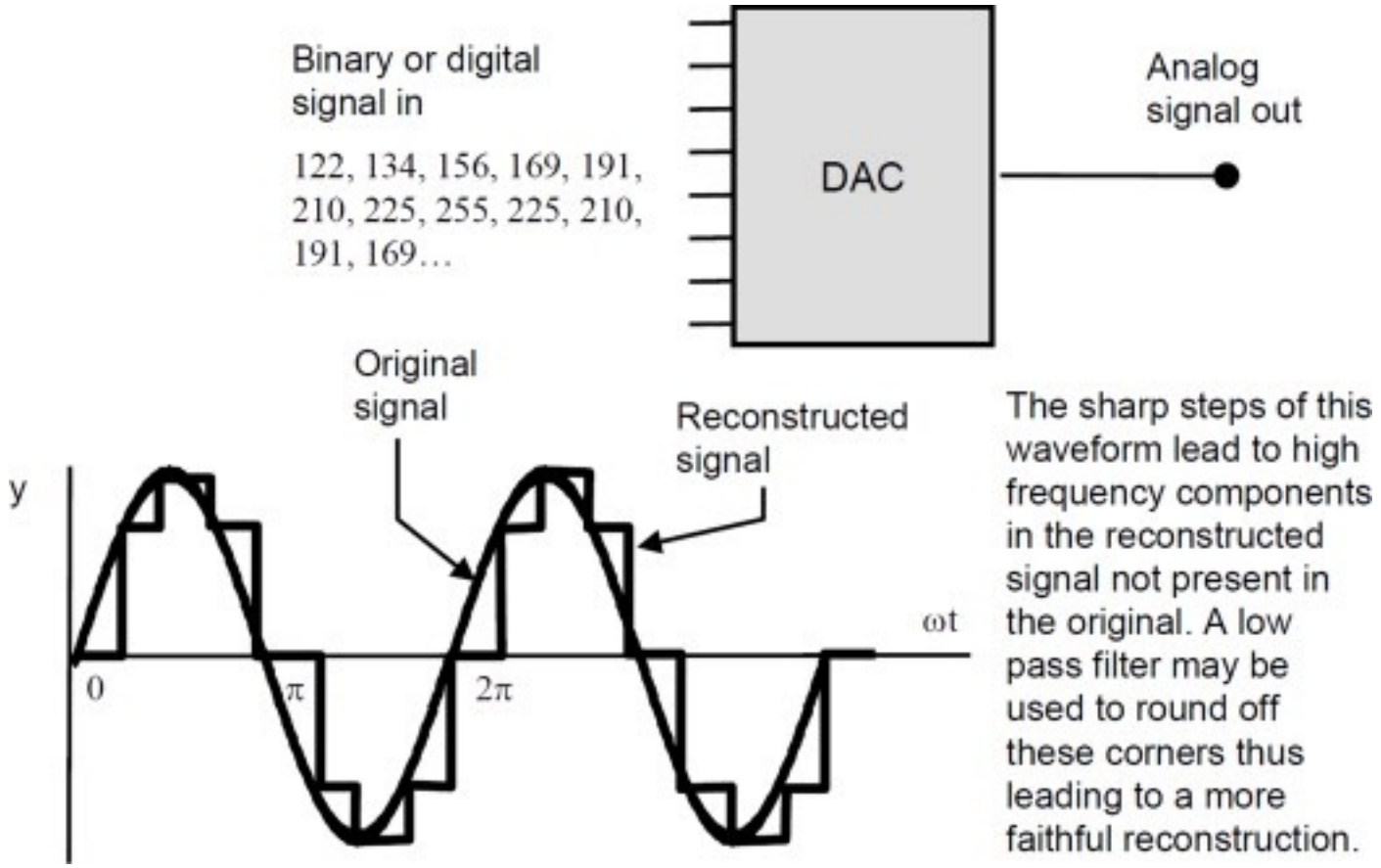
spi.write(buf)

Initialise the SPI bus with the given parameters:

- `baudrate` is the SCK clock rate.
- `polarity` can be 0 or 1, and is the level the idle clock line sits at.
- `phase` can be 0 or 1 to sample data on the first or second clock edge respectively.
- `bits` is the width in bits of each transfer. Only 8 is guaranteed to be supported by all hardware.
- `firstbit` can be `SPI.MSB` or `SPI.LSB`.
- `sck`, `mosi`, `miso` are pins (machine.Pin) objects to use for bus signals. For most hardware SPI blocks (as selected by `id` parameter to the constructor), pins are fixed and cannot be changed. In some cases, hardware blocks allow 2-3 alternative pin sets for a hardware SPI block. Arbitrary pin assignments are possible only for a bitbanging SPI driver (`id` = -1).
- `pins` - WiPy port doesn't `sck`, `mosi`, `miso` arguments, and instead allows to specify them as a tuple of `pins` parameter.

https://docs.micropython.org/en/latest/library/machine.SPI.html

Digital to Analog Converters (DAC)



***DACs output voltage steps at a certain frequency to construct an analog signal**

Image credit - https://www.sciencedirect.com/topics/engineering/digital-to-analog-conversion

Digital-to-Analog Converters (DAC): LTC1661

LINEAR TECHNOLOGY

LTC1661
Micropower Dual
10-Bit DAC in MSOP

The LTC1661 integrates two accurate, serially address - able, 10-bit digital-to-analog converters (DACs) in a single tiny MS8 package.

- Each buffered DAC draws just 60µA total supply current
- capable of supplying DC output currents in excess of 5mA
- reliably driving capacitive loads up to 1000pF
- Sleep mode further reduces total supply current to a negligible 1µA

Important

PIN FUNCTIONS

$\overline{CS}$/LD (Pin 1): Serial Interface Chip Select/Load Input. When $\overline{CS}$/LD is low, SCK is enabled for shifting data on D_{IN} into the register. When $\overline{CS}$/LD is pulled high, SCK is disabled and the operation(s) specified in the control code, A3-A0, is (are) performed. CMOS and TTL compatible.

SCK (Pin 2): Serial Interface Clock Input. CMOS and TTL compatible.

D_{IN} (Pin 3): Serial Interface Data Input. Input word data on the D_{IN} pin is shifted into the 16-bit register on the rising edge of SCK. CMOS and TTL compatible.

REF (Pin 4): Reference Voltage Input. $0V \le V_{REF} \le V_{CC}$.

V_{OUTA}, V_{OUTB} (Pin 8, Pin 5): DAC Analog Voltage Outputs. The output range is

$$0 \le V_{OUTA}, V_{OUTB} \le V_{REF}\left(\frac{1023}{1024}\right)$$

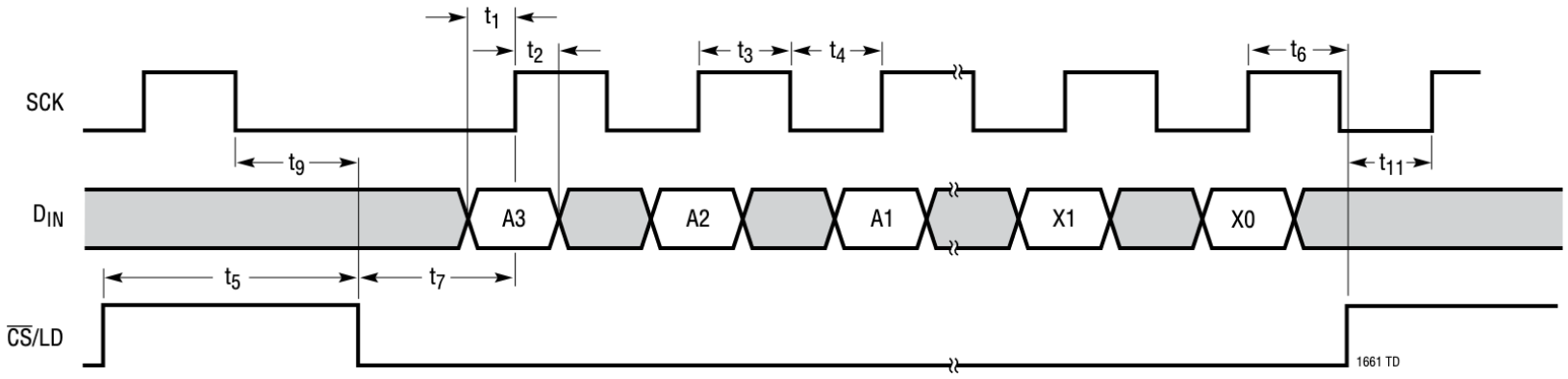
V_{CC} (Pin 6): Supply Voltage Input. $2.7V \le V_{CC} \le 5.5V$.

GND (Pin 7): System Ground.

https://www.mouser.com/datasheet/2/609/1661fb-1268166.pdf

Digital-to-Analog Converters (DAC): LTC1661

TIMING DIAGRAM



See Table 1. The 16-bit Input word consists of the 4-bit Control code, the 10-bit Input code and two don't-care bits.

Table 1. LTC1661 Input Word

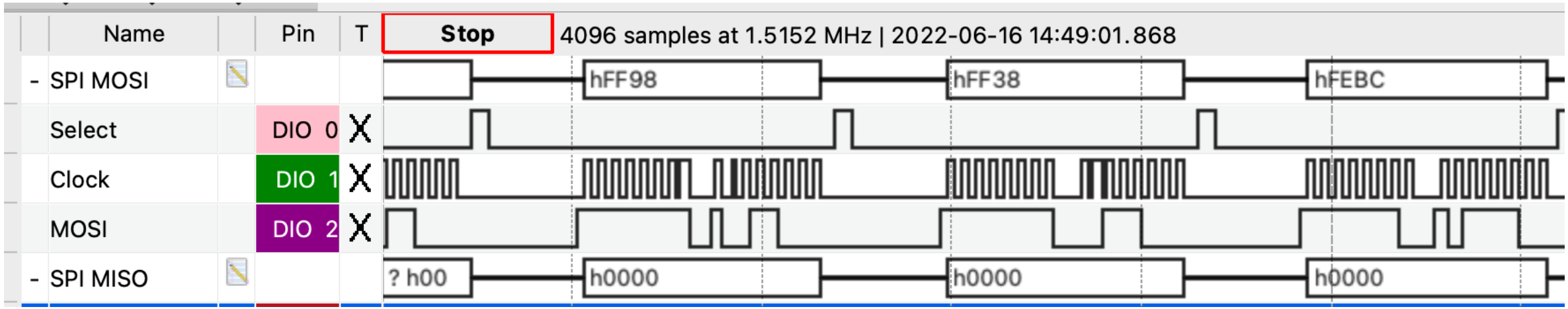
INPUT WORD															
A3	A2	A1	A0	D9	D8	D7	D6	D5	D4	D3	D2	D1	D0	X1	X0
CONTROL CODE				INPUT CODE										DON'T CARE	

- You will generate a 16 bit word that will get passed into the spi.write function. That word will contain the necessary control info and 10 bit number that will be one data point of the analog signal.
- Read DAC data sheet for control info

Microcontroller SPI Communication to External DAC: Assignment

1. Design a function generator
   - That outputs:
     - a sine wave, a square wave, a triangle wave and, a saw tooth wave
     - from 10 Hz to 100 Hz
     - from 0 V to 5 V, variable amplitude
   - The waveform will be selected through the use of two switches, where each of the four combinations of switch positions will correspond to one of the four waveforms.
   - There will be two rotary controls, one controlling the frequency and the other controlling the output voltage amplitude.
   - The output waveforms must be comprised of at least 25 samples per cycle.
   - The frequency and output amplitude must not change when switching between the four waveform types
   - The output voltage will always have a minimum voltage of 0 V and a variable maximum value to 5 V.
   - You must use the hardware SPI interface built into your MCU to connect to and communicate with an external digital-to-analog integrated circuit (LTC1661) to generate your waveforms

Trouble Shooting: DAD board Logic Analyzer



Time taken by processor before next byte is sent

- MicroPython inherently limits the frequency of the output analog signal…
- Experiment with the baudrate, number of samples in the waveform period, and delay (sleep) commands to achieve required frequency range.

References

Pico

- MicroPython Library Modules:
- https://docs.micropython.org/en/latest/library/machine.SPI.html
- https://docs.micropython.org/en/latest/rp2/quickref.html
- Rpi_PiPico_Guide: Chapter 10

DAC

- Data sheet: https://www.analog.com/media/en/technical-documentation/data-sheets/1661fb.pdf

Microcontrollers are mulitfunctional!